

AI-BDM

Architecture & Concepts Guide — a complete, plain-language handover.

This document is written for someone taking ownership of, reviewing, or extending this project **without** a background in web scraping or machine learning. Every technical term is explained in plain English the first time it appears, before the abbreviation or code-level detail. By the end, you should understand not just what the system does, but why each piece exists and how it actually works under the hood.

Two companion documents exist alongside this one: **AI-BDM_Project_Walkthrough.pdf** (a short, hands-on "try it yourself" guide) and **AI-BDM_QA_Test_Plan.xlsx** (142 detailed test cases). This document is the deepest of the three — read it when you need to actually understand the system, not just operate it.

Contents

- 1. What This System Does
- 2. Core Concepts, Explained in Plain English
- 3. The Technology Stack
- 4. The End-to-End Pipeline (All 8 Stages)
- 5. How and Where Data Is Stored
- 6. The Two Ways to Run This System
- 7. How It Connects to the Wider Project
- 8. What's Not Built Yet (Known Limitations)
- 9. Glossary — Every Term, One Place

1. What This System Does

AI-BDM is a research automation system. You give it one plain-English sentence describing the kind of business you're looking for — for example, *"find 3 dental clinics in Islamabad with no online booking"* — and it does the research a human analyst would otherwise do by hand: search the web, visit real company websites, read them, cross-check the businesses are real and currently operating, look at customer reviews, and hand back a clean, evidence-backed list.

"BDM" stands for Business Development / Business Data Management — this is built for B2B lead generation: finding real companies that match a specific description, the way a sales or marketing team would research prospects.

Why this is hard to do well (and why it's not "just scraping")

- Understanding a loosely-worded request the way a person would, not just matching keywords.
- Telling a real, currently-operating business apart from a defunct one, a directory listing, or an unrelated company that happens to share a name.
- Reading a website and answering a specific yes/no question about it honestly — saying "the evidence isn't there" instead of guessing.
- Doing all of this at scale, reliably, without quietly caching a network hiccup as if it were a real "this business doesn't exist" answer.

2. Core Concepts, Explained in Plain English

Read this section once, in order — every later section assumes you understand these terms.

Artificial Intelligence / LLM (Large Language Model)

A computer program trained on enormous amounts of text that can read a piece of text and produce a sensible response — summarize it, answer a question about it, extract structured information from it, or decide something about it. Think of it as a very well-read assistant that can follow written instructions, rather than a search engine that just matches keywords.

In this project: This project uses Groq as its LLM provider (a company that runs open large language models very fast). Two model sizes are used: a smaller/cheaper one (gpt-oss-20b) for quick, simple decisions (like "does this look like a dental clinic?"), and a larger one (gpt-oss-120b) for anything requiring real reasoning (like planning the whole research request, or answering a specific question about a business). A third model (qwen3.6-27b) is kept as a backup that automatically takes over if the primary model fails or times out.

NLP (Natural Language Processing)

The general field of getting computers to understand and work with human language — reading text and doing something useful with it. An LLM is one (very powerful, modern) way of doing NLP; the term NLP is broader and also covers older, simpler techniques.

In this project: Every step that reads your plain-English query, or reads a business's website text and reasons about it, is an NLP task, handled here by the LLM.

Web Scraping

Automatically visiting a webpage and reading its content, the same way a browser does when a person visits it — except done by code, at scale, and without a human clicking anything.

In this project: Two tiers exist here. A cheap, fast "native" fetch works for most ordinary websites. Some sites actively try to block automated visitors (Cloudflare challenges, CAPTCHAs) — for those, a paid third-party service called ScrapingBee is used, which can render JavaScript and disguise the request as a real browser. If a site fails twice in a row, the system stops paying for the expensive option on that site and falls back to the cheap one for its remaining pages.

Embeddings ("meaning as numbers")

A way of converting a piece of text into a list of numbers (a "vector") that captures its *meaning*, not just its exact words. Two sentences that mean similar things end up with similar number-lists, even if they don't share a single word in common. This is what lets a computer search "by meaning" instead of by exact text match.

In this project: A small, free, locally-run model called all-MiniLM-L6-v2 turns every chunk of scraped website/review text into a 384-number vector. It runs on the same machine, no internet call needed, and is fast enough to embed thousands of text chunks in seconds.

Vector Search / Vector Database

Once text is turned into number-lists (embeddings), you need a way to quickly find "which stored piece of text has a meaning closest to this new question?" among potentially thousands of candidates. A vector database is a storage system built specifically for that kind of similarity search, much faster than comparing every single one by hand.

In this project: This project uses Chroma, a lightweight, locally-run (no external server, no cost) vector database, stored in a folder called rag/.chroma/.

BM25 (keyword search)

A decades-old, non-AI search technique that ranks text by how well its exact words match a query — the same basic idea behind classic search engines before AI-based search existed. Not "smart" about meaning, but fast, reliable, and good at exact terms (like a specific product name).

In this project: Used *alongside* embeddings (a "hybrid" search) — combining exact keyword matching with meaning-based matching catches more relevant results than either alone.

RAG (Retrieval-Augmented Generation)

A pattern for using an LLM more reliably: instead of asking the LLM to just answer from what it "remembers" (which risks it confidently making something up — called "hallucinating"), you first *retrieve* the actual, real, relevant text from your own data, and then ask the LLM to *generate* its answer using only that retrieved text as evidence. The LLM is instructed to say "not enough evidence" rather than guess when the retrieved text doesn't actually answer the question.

In this project: Every specific-question answer this system gives (e.g. "does this business have online booking?") is grounded this way: the relevant scraped page text is retrieved first (via the hybrid BM25 + embeddings search above), and the LLM is only allowed to answer from that real text, always citing which page it came from.

API (Application Programming Interface)

A defined way for two separate computer programs to talk to each other over a network, using structured requests and responses instead of a human clicking a website. Think of it as a menu of specific things another program is allowed to ask this one to do.

In this project: This project exposes a REST API (the most common API style, built on standard web requests) so other systems — like a Laravel backend — can trigger a research run and read results programmatically instead of running the command-line tool by hand.

Synchronous vs. Asynchronous ("async"), and Background Jobs

Synchronous means "the caller waits until it's completely done" — like standing at a counter until your order is ready. Asynchronous means "the work starts, and the caller can check back later or get notified" — like taking a buzzer at a restaurant and being told when your table's ready. A background job is work that runs this second way, without blocking anything else.

In this project: A full research request can take several minutes. This project offers both styles: a simple "wait for it" endpoint for quick manual use, and a "submit and poll for progress" job system for real integrations where nothing should sit there frozen for minutes at a time.

Idempotency ("safe to repeat")

A fancy word for a simple, important idea: doing the same operation twice should have the exact same effect as doing it once — not create a duplicate. This matters because networks are unreliable; if a caller doesn't get a response and retries "just in case," idempotency guarantees that retry doesn't accidentally run the (possibly expensive) operation a second time.

In this project: A caller can attach a unique "Idempotency-Key" to a request; if the same key is sent again, the system returns the exact original result instead of doing the work over.

3. The Technology Stack

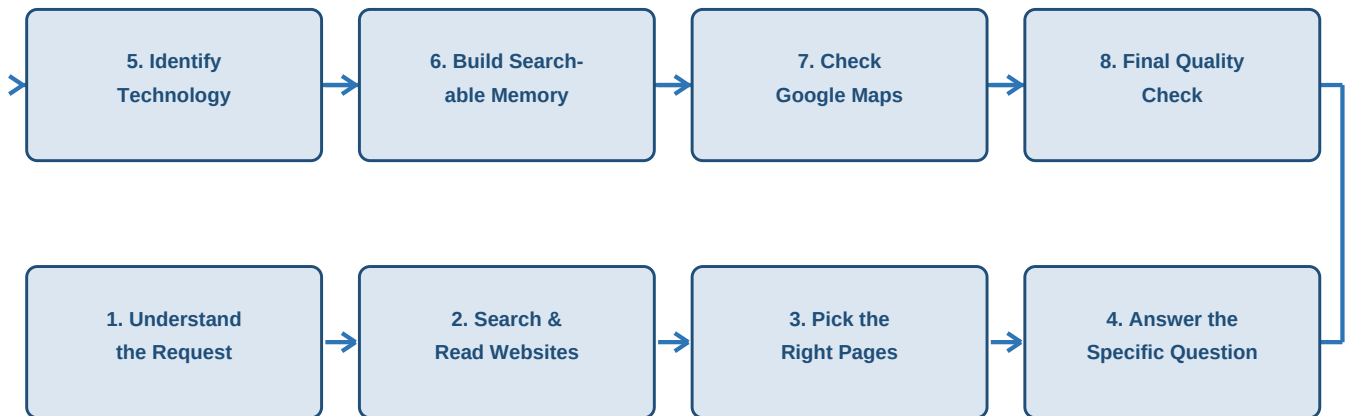
Everything the system is built from, and in plain terms, why each piece is there.

Python 3.12	Language The programming language the whole system is written in — chosen for its strong ecosystem of both web-scraping and machine-learning libraries.
FastAPI + Uvicorn	Web framework FastAPI defines the API's request/response shapes and validates them automatically; Uvicorn is the actual server process that keeps the API running and listening for requests.
Pydantic	Data validation Defines the exact shape every request and response must have (which fields, what type, what's required) and automatically rejects anything that doesn't match — catches bad input before it reaches the real logic.
Groq (hosted LLMs)	AI / language understanding Runs the large language models that understand requests, plan research, and answer specific questions from evidence. See the LLM concept above.
Serper.dev	Web search A paid API that provides Google Search and Google Maps/Places results programmatically — the system's way of "searching Google" and "looking up a business on Maps" without a browser.
Python's requests library	Scraping (standard) Fetches ordinary, non-protected web pages directly and cheaply.
ScrapingBee	Scraping (protected sites) A paid service that renders JavaScript and disguises requests as a real browser, used only when the cheap method is blocked.
Apify (managed scraping actors)	Review harvesting A platform that runs pre-built "scraping robots" for specific sites (Google Maps reviews, Reddit) that are otherwise very hard to scrape directly.
Wappalyzer (a vendored, patched copy)	Technology fingerprinting Identifies what software (CMS, booking system, analytics, etc.) a business's website is built on, by recognizing patterns in its HTML/cookies/headers.
sentence-transformers (all-MiniLM-L6-v2)	Meaning-based search The small local model that turns text into "meaning numbers" (embeddings) — see the Embeddings concept above.
Chroma	Vector storage/search Stores those embeddings and finds the closest matches quickly — see the Vector Search concept above.
rank-bm25	Keyword search The classic keyword-ranking algorithm used alongside embeddings for hybrid search.
CSV files + a plain folder structure	Local data storage Per-business results and per-page crawl records are stored as CSV spreadsheets; each business's raw scraped text and reviews live in its own folder.
Cloudflare R2 (via boto3)	Optional remote storage An optional, S3-compatible cloud storage service that can mirror the local per-business folders off-machine, for backup/durability.

pytest	Testing The framework that runs this project's 115+ automated tests, checking real behavior without needing real API keys for most of them.
pip + a virtual environment (venv)	Packaging / environment Standard Python tooling for installing exact, pinned dependency versions in an isolated environment, so the project behaves the same on any machine.
Docker	Containerization Packages the whole service (code + Python + system dependencies) into a single, portable unit that runs identically on any machine with Docker installed — used for the production/integration deployment.

4. The End-to-End Pipeline (All 8 Stages)

One research request flows through up to 8 stages, always in this order. Stages 4, 5, 6, and 7 only run when the request actually needs them — a plain "find 3 dental clinics in Islamabad" with no extra condition skips straight from Stage 2 to Stage 8.



Steps 4, 5, 6, and 7 only run when the request actually needs them.

Stage 1 — Understand the Request

Turns your one plain-English sentence into a structured plan — the same way a human research assistant would first make sure they understood exactly what you're asking for before starting.

How it actually works:

- The LLM reads the sentence and extracts: what industry, what location, how many results you want, whether there's an extra condition to check ("...with no online booking"), and whether you named one specific company instead of asking for a general search.
- A safety check ("moderation") runs first, before anything else — if the request violates usage policy, the system stops immediately, before spending any money on search/scraping.
- If the request is too vague to act on confidently, the system stops and asks a clarifying question instead of guessing — up to 3 rounds before it proceeds with its best interpretation regardless.

Built with: The Groq LLM (the larger model, since this needs real judgment, not just simple classification).

What comes out: A structured plan (industry, location, count, the extra condition if any) that every later stage reads.

Stage 2 — Search & Read Real Websites

The actual research: finding real candidate businesses and visiting their websites, the way a person would open browser tabs and read pages — except automated, and checking each one against the plan from Stage 1.

How it actually works:

- Searches the web (via Serper.dev, which wraps Google Search) for candidate businesses matching the industry and location.
- Visits each candidate's website and reads its content (web scraping — see the concept section). Cheap fetch first; the paid anti-bot-resistant option only if that's blocked.
- A fast relevance check filters out obvious non-matches (wrong industry, wrong location, clearly not a real business) before spending more effort on a candidate.
- If the request specified a target number of results ("give me 20..."), the system keeps searching — deeper search pages, related sites — until it genuinely finds that many real matches or exhausts reasonable options, rather than padding the count with weak matches.
- Also harvests customer reviews for each qualified business where available (Google reviews, Reddit mentions).

Built with: Serper.dev (search), the scraping tools above, Apify (reviews).

What comes out: A committed record per real, qualifying business: its website text, contact details, and any reviews found.

Stage 3 — Pick the Right Pages to Focus On

A business's website might have 20+ pages. Before trying to answer a specific question about it, the system figures out which few pages are actually likely to contain the answer, instead of re-reading everything every time.

How it actually works:

- Uses the hybrid keyword + meaning-based search (BM25 + embeddings) described in the concepts section to rank the business's own already-scraped pages by relevance to the question.
- Only falls back to asking the LLM to guess which page is relevant in the rare case the automatic search finds nothing useful — the common case needs no extra AI call at all.

Built with: The hybrid search described above (BM25 + embeddings), with an LLM fallback.

What comes out: A short list of the most relevant pages for the next stage to actually read in depth.

Stage 4 — Answer the Specific Question

Only runs when the request had an extra condition to check (like "...with no online booking"). Reads the relevant page text and gives a direct, honest answer — grounded in what was actually found, using the RAG pattern described in the concepts section.

How it actually works:

- The LLM is given the actual text from the pages Stage 3 picked, and asked the specific question, with strict instructions to answer only from that text.
- If the evidence genuinely isn't there, the answer says so explicitly ("not enough evidence") instead of guessing — this is the core anti-hallucination safeguard.
- Every answer cites exactly which page(s) it was grounded in.

Built with: The Groq LLM (larger model), using the RAG pattern.

What comes out: A direct answer to the specific question, with a confidence level and the exact source pages cited.

Stage 5 — Identify the Website's Technology

Only runs when the request cares about a business's technology (e.g. "...using WordPress"). Figures out what platform, booking system, or marketing tools a website actually runs on.

How it actually works:

- Examines the page's HTML, cookies, response headers, and DNS records for known technology "fingerprints" — recognizable patterns that reveal what software built the site.
- Four separate detection methods run independently and are merged, so one method failing doesn't blank out the whole result.

Built with: Wappalyzer (a vendored, bug-patched copy — see Section 8 for why it's vendored, not a normal install).

What comes out: A profile of detected technologies (CMS, booking tools, analytics, etc.) for that business.

Stage 6 — Build a Searchable Memory of Everything Found

Organizes everything read in Stage 2 (website text and reviews) into the meaning-based search index described in the concepts section, so Stage 3/4 can quickly find the right evidence instead of re-reading everything from scratch every time.

How it actually works:

- Breaks scraped text into small chunks, and turns each chunk into an embedding (a "meaning number-list") using the local embedding model.
- Stores those embeddings in the local vector database (Chroma), keeping website content and review content in two separate, independently-ranked groups, so one doesn't drown out the other.

Built with: sentence-transformers (embedding model) + Chroma (vector database).

What comes out: A searchable local index of everything found for that business, ready for fast meaning-based lookup.

Stage 7 — Cross-Check Against Google Maps

Confirms each business is real and currently operating by matching it against its actual Google Maps listing — catching cases where the website is stale, the business closed, or the match is simply wrong.

How it actually works:

- Looks the business up via Serper.dev's Google Places integration.
- Tries to confirm the match first by matching the physical address, then by phone number if the address alone isn't distinctive enough, and only falls back to matching by name alone when there's exactly one unambiguous candidate — each tier is recorded so you know how confident the match is.

Built with: Serper.dev (Google Places/Maps).

What comes out: Confirmed rating, review count, and category from Google Maps, plus a confidence label for the match.

Stage 8 — Final Quality Check

Before handing anything back, the system re-checks its own work — the same instinct a careful analyst has to double-check their findings before submitting them.

How it actually works:

- Checks that phone numbers, emails, addresses, and ratings are validly formatted.
- Checks that the business's name is consistent across every source (its own site, Google Maps, review platforms) — catching a case where the data actually belongs to a different, similarly-named business.
- Checks that harvested reviews genuinely reference this specific business, not something unrelated that happened to get picked up.
- Anything questionable is flagged in a report, never silently hidden or silently dropped.

Built with: Rule-based checks — no AI involved in this stage, deliberately, since consistency-checking doesn't need judgment, just comparison.

What comes out: A quality report listing anything worth a second look, alongside the final results.

5. How and Where Data Is Stored

No traditional database (like PostgreSQL or MySQL) is used inside this Python service itself — everything is stored as plain files, which keeps the system simple and makes every result directly inspectable without needing a database client.

crawl_index.csv	One row per web page crawled — the most detailed record, spreadsheet-style.
leads_clean.csv	One row per business (rolled up from all its pages) — the main results file.
leads_with_maps.csv	The same rollup, with Google Maps confirmation data added.
accuracy_report.txt	The Stage 8 quality-check output — anything flagged as questionable.
storage/<business-domain>/	One folder per business, holding the actual scraped page text, harvested reviews, and technology profile — the raw evidence behind every result.
rag/.chroma/	The local vector database from Stage 6 — can always be safely deleted and rebuilt from the storage/ folders.

Optionally, everything in storage/ can also be mirrored to Cloudflare R2 (cloud object storage) for an off-machine backup copy — this is disabled unless specifically configured.

6. The Two Ways to Run This System

1. Command line (for manual/direct use)

```
python main.py --query "find 3 dental clinics in Islamabad with no online booking"
```

Runs the full 8-stage pipeline directly in a terminal and writes results to the CSV files described above. This is the simplest way to actually see the system work.

2. REST API (for another program to call it)

```
python -m uvicorn api:app --port 8000
```

Starts a web server exposing the same functionality over HTTP, so another system (like the Laravel backend) can trigger a research run and read results programmatically. Two styles exist within the API:

- **A simple, synchronous endpoint** (POST /pipeline/run) — the caller sends a request and waits (potentially several minutes) for the complete result. Simple, but ties up the caller the whole time.
- **An asynchronous job system** (POST /api/v1/jobs and related endpoints) — the caller submits a request, gets an ID back immediately, and can check progress or cancel it, without being frozen waiting. Built for real integrations where nothing should sit blocked for minutes. See the async/background job concept in Section 2.

7. How It Connects to the Wider Project

This Python service is one piece of a larger system with three separate parts, each with a clear, deliberately-enforced boundary:

Next.js frontend	The actual website/app a user sees and clicks around in. It never talks to this Python service directly.
Laravel backend	The system of record — handles user accounts, organizations, approvals, and is the <i>only</i> part allowed to send outbound email. It calls this Python service to run research, then imports the results into its own database.
This Python service	Does the actual research work described in this document, and nothing else — it never sends email, never decides who's authorized to do what, and is never called directly by the frontend.

This separation is deliberate: research (which can be slow, occasionally wrong, and depends on unpredictable external websites) is kept isolated from anything that touches real user accounts or sends real outbound communication.

All three parts are packaged with Docker (a way of bundling an application with everything it needs so it runs identically on any machine) so a developer can start the entire backend stack with one command, without installing PHP, Python, or a database directly on their own machine.

8. What's Not Built Yet (Known Limitations)

Stated honestly and specifically, rather than left for someone to discover the hard way:

- **No non-AI fallback exists.** Every request genuinely requires the LLM (Groq) to be reachable — there's no simpler rule-based backup path for understanding a request.
- **Some review platforms actively block automated access** (Yelp, Trustpilot) even with the paid anti-bot tool — a production fix would need each platform's own official paid API.
- **A vendored, hand-patched copy of Wappalyzer is used** instead of a normal install, because the publicly published version of that package was found to silently serve different, incompatible code over time under the same version number — a real reproducibility risk that's been permanently worked around by keeping a known-good copy directly in this project.
- **The website's robots.txt "do not crawl" rules are not currently enforced** — they're read only to find a site's page listing, not to respect crawling restrictions. A real, acknowledged gap, not a design choice.
- **No content is cryptographically verified** (no checksums/hashes) on any stored file yet.
- **The asynchronous job system keeps its state only in memory** — if the service restarts, any in-progress or recently-finished job information is lost (though anything already saved to the storage/folders is safe).
- **"Resuming" a failed job restarts the search from the beginning**, not from the exact point it stopped — it just avoids redoing work for businesses already found and saved, rather than picking up mid-search.

The full, exhaustive list — including gaps specific to individual stages — lives in `docs/KNOWN_LIMITATIONS.md` and `docs/backend-handoff/KNOWN_LIMITATIONS.md` in the project repository.

9. Glossary — Every Term, One Place

Term	Meaning
AI / LLM	A large language model — a program that reads and generates text intelligently. See Section 2.
API	A defined way for two programs to talk to each other over a network. See Section 2.
Async / Background job	Work that runs without freezing the caller — submit now, check later. See Section 2.
BM25	A classic, non-AI keyword search ranking algorithm. See Section 2.
Embedding	Text converted into a list of numbers that captures its meaning. See Section 2.
Hallucination	When an AI model confidently states something false or made-up. RAG (below) is the main defense against this.
Idempotency	Doing an operation twice has the same effect as doing it once — "safe to repeat." See Section 2.
NLP	Natural Language Processing — the broad field of computers working with human language. See Section 2.
Pipeline	The fixed sequence of stages a request goes through, described in Section 4.
RAG	Retrieval-Augmented Generation — look up real facts first, then answer using only those facts. See Section 2.
REST API	The most common style of web API, built on standard HTTP requests (GET, POST, etc.).
Scraping	Automatically reading a webpage's content, the way a browser would for a person. See Section 2.
Synchronous	The caller waits until the work is completely finished before getting a response.
Vector database	Storage built specifically for fast "find the closest meaning" search over embeddings. See Section 2.